

Chapter 2. Numerical methods for solving linear systems

ARIN7013 Numerical Methods in Artificial Intelligence

The University of Hong Kong



Objective

Numerical method for solving linear systems (Jacobi Method, GaussSeidel method, Cholesky decomposition, singular value decomposition (SVD), low-rank matrix approximation, with applications in artificial intelligence and machine learning).

Table of Contents

1	Motivation	4
2	Direct Method	13
	■ Forward/backward substitution	15
	■ Gaussian elimination method	22
	■ LU factorization	33
	■ Stability analysis	49
3	Sparse matrix and iterative solvers	62
	■ sparse matrix	63
	■ What is a sparse matrix?	
	■ How to store a sparse matrix? (optional+self-study)	
	■ Splitting-based Iterative Methods	74
	■ Jacobi iterative method	
	■ Gauss-Seidel iterative method	
4	Singular Value Decomposition (SVD) and Low-rank approximation	93
	■ Definition and Computation	94
	■ Application: Image Compression with SVD	117
	■ Application: Matrix Completion with Low-rank approximation	124
5	Conclusion	129

2.1 Motivation

Numerical Linear Algebra is central to Machine Learning

Example (Probability theory: Normal Distribution)

$$X \sim \mathcal{N}(\mu, \Sigma)$$
$$p_X(x) = \frac{1}{\sqrt{(2\pi)^n \det(\Sigma)}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

question

- How to calculate $\det(\Sigma)$ and Σ^{-1} efficiently when the dimension d is large?
- How to calculate matrix vector multiplication efficiently?

Numerical Linear Algebra is central to Machine Learning

Example (Kernel Ridge Regression (KRR))

- Let $\mathcal{H}$ be a RKHS, which is associated with a p.d. kernel K on $\mathcal{X}$
- KRR is obtained by regularizing the MSE criterion by the RKHS norm,

$$\hat{f} \in \arg \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i) + \lambda \|f\|_{\mathcal{H}}^2. \quad (1)$$

Solution

One solution is

$$\hat{f}(x) = \sum_{i=1}^n \alpha_i K(x_i, x), \text{ with } \alpha = (\mathbb{K} + \lambda n \mathbb{I})^{-1} \mathbf{y}.$$

Numerical Linear Algebra is central to Machine Learning

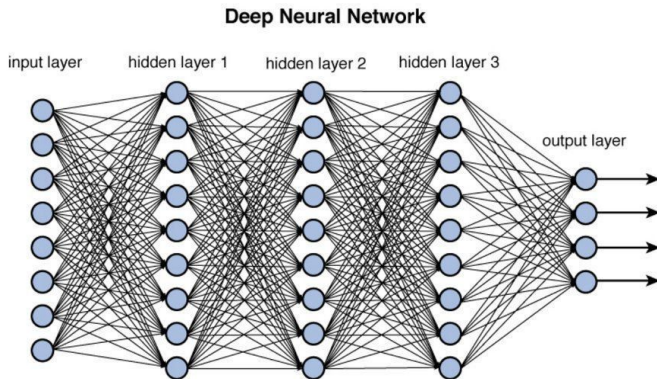
Example (Principal Component Analysis (PCA))

$$\mathbf{X}^T \mathbf{X} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T \Rightarrow \mathbf{x} \mapsto \mathbf{Q}_{1:k}^T \mathbf{x}$$



Labeled Faces in the Wild (LFW): Huang et al., 2009. Pre-processed by Samira Samadi

DNN architecture¹



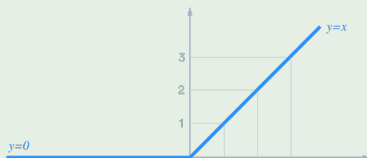
¹<https://towardsdatascience.com/training-deep-neural-networks-9fdb1964b964>

Numerical Linear Algebra is central to Machine Learning

Example (Neural Network 1/2)

■ Activation function

$$\sigma(x) = (x)_+ = \max(x, 0), x \in \mathbb{R}.$$



- Number of hidden layers or depth is denoted by L . Let $\mathbf{p} := (p_0, \dots, p_{L+1}) \in \mathbb{R}^{L+2}$ with $p_0 := d$ be the width of the network.
- Shifted activation function $\sigma_{\mathbf{v}} : \mathbb{R}^r \rightarrow \mathbb{R}^r$ by

$$\sigma_{\mathbf{v}}(\mathbf{y}) := (\sigma(y_1 + v_1), \dots, \sigma(y_r + v_r))$$

Example (Neural Network 2/2)

- Deep multilayer ReLU network Φ with input dimension d and L layers is a sequence of matrix-vector tuples

$$\Phi = ((\mathbb{A}_1, \mathbf{v}_1), (\mathbb{A}_2, \mathbf{v}_2), \dots, (\mathbb{A}_{L+1}, \mathbf{v}_{L+1}))$$

with $\mathbb{A}_\ell \in \mathbb{R}^{p_\ell \times p_{\ell-1}}$ for all $\ell \in \mathbb{N}_+$.

- The associated realization of Φ with activation function σ is $R_\sigma(\Phi) : \mathbb{R}^d \rightarrow \mathbb{R}^{p_L}$, defined by

$$R_\sigma(\Phi)(\mathbf{x}) = \mathbb{A}_{L+1} \circ \sigma_{\mathbf{v}_L} \circ \dots \circ \mathbb{A}_2 \circ \sigma_{\mathbf{v}_1} \circ \mathbb{A}_1 \mathbf{x} + \mathbf{v}_{L+1} \quad \text{for all } \mathbf{x} \in \mathbb{R}^d.$$

One can use

$$\mathcal{V} := \{R_{\sigma}(\Phi)(\mathbf{x}) : \Phi = ((\mathbb{A}_1, \mathbf{v}_1), (\mathbb{A}_2, \mathbf{v}_2), \dots, (\mathbb{A}_{L+1}, \mathbf{v}_{L+1}))\}$$

as the **hypothesis space** to complete a learning task in the supervised learning.

Let $\{(\mathbf{x}_i, y_i) : i = 1, \dots, n\}$ be the training data, then one can solve the following minimization problem to obtain the solution,

$$\hat{f} \in \arg \min_{f \in \mathcal{V}} \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i) + \lambda \|f\|_{\mathcal{V}}^2. \quad (2)$$

Fundamental Operations of Numerical Linear Algebra for Machine Learning

■ Efficient Matrix-Vector Multiplication

$$\mathbf{v} \mapsto \mathbf{A}\mathbf{v}$$

Examples: deep neural networks, KRR

■ Solution of Linear Systems

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

Examples: Gaussian density, linear regression, Galerkin method, etc.

■ Matrix Decomposition

$$\mathbf{A} = \mathbf{L}\mathbf{U} \quad \mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^* \quad \mathbf{A} = \mathbf{Q}\mathbf{R}$$

Examples: (kernel) PCA, Gaussian Process inference, etc.

2.2 Direct Method

Applications of linear equations

- In mathematics, a **system of linear equations** (or linear system) is a collection of linear equations involving the same set of variables.
- In mathematics, the theory of linear systems is the basis and a fundamental part of linear algebra, a subject which is used in most parts of modern mathematics.
- Computational algorithms for finding the solutions are an important part of numerical linear algebra, and play a prominent role in **engineering, physics, chemistry, computer science, and economics**.
- A system of **non-linear equations** can often be approximated by a linear system, a helpful technique when making a **mathematical model** or **computer simulation** of a relatively complex system.

1. Forward/backward substitution

Linear equation system

- Consider solving a linear system of the form:

$$\left\{ \begin{array}{lcl} E1 : a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n & = & b_1, \\ E2 : a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n & = & b_2, \\ \vdots & & \vdots \\ Em : a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n & = & b_m. \end{array} \right. \quad (3)$$

where $x_1, x_2, \dots, x_n$ are unknowns.

- When $m > n$, the system is **overdetermined**; When $m < n$, the system is **underdetermined**;
- In this section, we consider only the **square system**, i.e., $m = n$.

Linear equation system

- Matrices are useful devices for representing systems of equations. Thus, the above system can be written as

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix}$$

- Then we denote these matrices by $\mathbb{A}$, $\mathbf{x}$, and $\mathbf{b}$, so that the equation becomes $\mathbb{A}\mathbf{x} = \mathbf{b}$. We assume $\mathbb{A}$ is an **invertible (nonsingular)** matrix. So the system has unique solution.
- Because we use a **finite precision** floating point system, some algorithms may produce wrong answers, particularly when $\mathbb{A}$ is **near singular**.
- Our motivation is to design efficient and reliable algorithms for computers to solve the problem.

Elementary operations

We use three **row elementary operations** to simplify the linear system:

- 1 Interchanging two equations in the system: $E_i \leftrightarrow E_j$
- 2 Multiply an equation by a nonzero number $\lambda \neq 0$: $\lambda E_i \rightarrow E_i$
- 3 Adding to an equation a multiple of some other equation:
 $E_i + \lambda E_j \rightarrow E_i$

Theorem (Equivalent systems)

If one system of equations is obtained from another by a finite sequence of elementary operations, then the two systems are equivalent.

In linear algebra, a **triangular matrix** is a special kind of square matrix.

- A square matrix $\mathbb{L}$ is called **lower triangular** if all the entries above the main diagonal are zero.
- Similarly, a square matrix $\mathbb{U}$ is called **upper triangular** if all the entries below the main diagonal are zero.
- A matrix that is both upper and lower triangular is called a **diagonal matrix**.

Forward and back substitution

- A matrix equation in the form $\mathbb{L}\mathbf{x} = \mathbf{b}$ or $\mathbb{U}\mathbf{x} = \mathbf{b}$ is easy to solve by **forward substitution** for lower triangular matrices and analogously **back substitution** for upper triangular matrices.
- The matrix equation $\mathbb{L}\mathbf{x} = \mathbf{b}$ can be written as

$$\begin{array}{rcll} \ell_{1,1}x_1 & & & = b_1 \\ \ell_{2,1}x_1 + \ell_{2,2}x_2 & & & = b_2 \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{n,1}x_1 + \ell_{n,2}x_2 + \cdots + \ell_{n,n}x_n & = & b_n \end{array}$$

- Observe the first equation $\ell_{1,1}x_1 = b_1$ only involves x_1 , thus one can solve for x_1 directly. The second equation only involves x_1 and x_2 , and thus can be solved once one gets x_1 . Similarly, we get

$$x_1 = \frac{b_1}{\ell_{1,1}}, \quad x_2 = \frac{b_2 - \ell_{2,1}x_1}{\ell_{2,2}}, \dots, x_n = \frac{b_n - \sum_{i=1}^{n-1} \ell_{n,i}x_i}{\ell_{n,n}}.$$

- What is the computational complexity?

The computational complexity of forward substitution

Let us compute the operations we need to finish the forward substitution.

$$x_1 = \frac{b_1}{\ell_{1,1}},$$

$$x_2 = \frac{b_2 - \ell_{2,1}x_1}{\ell_{2,2}},$$

$$x_3 = \frac{b_3 - \ell_{3,1}x_1 - \ell_{3,2}x_2}{\ell_{3,3}},$$

$\vdots$

$$x_n = \frac{b_n - \ell_{n,1}x_1 - \ell_{n,2}x_2 - \cdots - \ell_{n,n-1}x_{n-1}}{\ell_{n,n}}.$$

The number of operations: (1) Adding and subtracting: $\frac{n(n-1)}{2}$; (2)

Multiplication: $\frac{n(n-1)}{2}$; (3) Division: n .

In total, the computational complexity is n^2 .

Back substitution

The matrix equation $\mathbf{U}\mathbf{x} = \mathbf{b}$ can be written as

$$\begin{array}{ccccccc} u_{1,1}x_1 & + \cdots + & u_{1,n-1}x_{n-1} & + & u_{1,n}x_n & = & b_1 \\ u_{2,1}x_1 & + \cdots + & u_{2,n-1}x_{n-1} & & & = & b_2 \\ \vdots & & \vdots & & \ddots & & \vdots \\ u_{n,1}x_1 & & & & & = & b_n \end{array}$$

Question

Can you write down the back substitution algorithm to solve for $\mathbf{x}$? What is the computational complexity? n^2

2. Gaussian elimination method

Question

We have learned that linear equation system with triangular matrix can be efficiently solved. However, **how to reduce a general matrix into a triangular matrix?**

Gaussian Elimination (cont.)

Algorithm Forward elimination.

Main Step of the Gaussian elimination.

- 1 For $i = 1, \dots, n - 1$
- 2 For $j = i + 1, \dots, n$
- 3 Multiply i -th row by $-\frac{a_{ji}}{a_{ii}}$ and add to j -th row

Question

What is the computational complexity of the forward elimination?

A More Precise Description (Cont'd)

Thus

$$\tilde{A}^{(k)} = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} & \cdots & a_{1,k-1}^{(1)} & a_{1k}^{(1)} & \cdots & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & \cdots & a_{2,k-1}^{(2)} & a_{2k}^{(2)} & \cdots & a_{2n}^{(2)} \\ \vdots & & \ddots & & \vdots & \vdots & & \vdots \\ \vdots & & & \ddots & \vdots & \vdots & & \vdots \\ \vdots & & & & a_{k-1,k-1}^{(k-1)} & a_{k-1,k}^{(k-1)} & \cdots & a_{k-1,n}^{(k-1)} \\ \vdots & & & & 0 & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ \vdots & & & & \vdots & \vdots & & \vdots \\ 0 & \cdots & \cdots & \cdots & 0 & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{bmatrix}$$

Gaussian elimination procedure is described more precisely, although more intricately, by forming a sequence of augmented matrices $\tilde{A}^{(1)}$, $\tilde{A}^{(2)}$, $\dots$, $\tilde{A}^{(n)}$, where $\tilde{A}^{(1)}$ is the matrix $\tilde{A}$ given earlier and $\tilde{A}^{(k)}$, for each $k = 2, 3, \dots, n$, has entries $a_{ij}^{(k)}$, where:

$$a_{ij}^{(k)} = \begin{cases} a_{ij}^{(k-1)} & \text{when } i = 1, 2, \dots, k-1 \text{ and } j = 1, 2, \dots, n+1 \\ 0 & \text{when } i = k, k+1, \dots, n \text{ and } j = 1, 2, \dots, k-1 \\ a_{ij}^{(k-1)} - \frac{a_{i,k-1}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} a_{k-1,j}^{(k-1)} & \text{when } i = k, k+1, \dots, n \text{ and } j = k, k+1, \dots, n+1 \end{cases}$$

The computational complexity of Gaussian Elimination²

- The number of operations in **first column**, from 2nd row to nth row.
 - Division: $n - 1$
 - Multiplication: $(n - 1)^2$
 - Adding or subtracting: $(n - 1)^2$
- The number of operations in **second column**, from 3rd to nth row.
 - Division: $n - 2$
 - Multiplication: $(n - 2)^2$
 - Adding or subtracting: $(n - 2)^2$
- continue ...

Therefore, the computational complexity is $O(n^3)$:

$$\sum_{i=1}^{n-1} \left(\sum_{j=i+1}^n 1 + 2(n-i) \right) = 2n^3/3 - n^2/2 - n/6.$$

²The 'division' involved is for the calculation of the multiplier a_{ji}/a_{ii} for $i = 1, \dots, n-1$ and $j = i+1, 3, \dots, n$

How to solve a linear system?

Solve $\mathbb{A}\mathbf{x} = \mathbf{b}$ for a general matrix $\mathbb{A}$ by the Gaussian elimination method.

- 1 Apply the elementary row operations and transform $[\mathbb{A} \quad \mathbf{b}]$ into an upper triangular matrix $[\mathbb{U} \quad \mathbf{c}]$ Forward elimination
- 2 Solve $\mathbb{U}\mathbf{x} = \mathbf{c}$ by back substitution

Why we name it 'Gaussian elimination method'?

The solution of N linear equations in N unknowns by elimination was already known to ancient Chinese^a. Before Gauss many mathematicians in Eurasia were performing and perfecting it yet as the method became relegated to school grade, few of them left any detailed descriptions. Thus the name Gaussian elimination is only a convenient abbreviation of a complex history.

^aHart, Roger (2011), The Chinese Roots of Linear Algebra, Baltimore: Johns Hopkins, ISBN 978-0801897559.

Example

Let

$$\mathbb{A} = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix} \text{ and } \mathbf{b} = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix},$$

then solve for $\mathbf{x}$, such that $\mathbb{A}\mathbf{x} = \mathbf{b}$.

Proof.

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right] \rightarrow \left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 2 & 1 & 5 \end{array} \right] \rightarrow \left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 0 & -1 & 1 \end{array} \right]$$

Use backward substitution, we obtain $\mathbf{x} = [2; 3; -1]$. □

The computational complexity of the forward elimination is $2n^3/3 + n^2/2 - n/6$.

- Since the computational complexity of the back substitution is n^2 , the total complexity of the Gaussian elimination is $2n^3/3 - n^2/2 - n/6$.
- What happens if a_{ij} at i -th step is zero? **Gaussian elimination with pivoting.**

Pivoting: an example

- To demonstrate the basic idea, let us see an extreme example. Assume we have a computer with **1-digit** accuracy.

$$\begin{cases} 0.001x_1 + x_2 = 3, \\ x_1 + 2x_2 = 5 \end{cases} \quad (4)$$

- Forward elimination: $-998x_2 = -2995$
- Rounding: $x_2 = 3$
- Back substitution to first equation: $x_1 = 0$
- Exact solution is $x_1 = -1.002$, $x_2 = 3.001$.
 - Relative error of x_2 is 0.3×10^{-3}
 - Relative error of x_1 is **1**.

Alternative formulation

- Exchange the order of two equations

$$\begin{cases} x_1 + 2x_2 = 5, \\ 0.001x_1 + x_2 = 3 \end{cases} \quad (5)$$

- Forward elimination: $998x_2 = 2995$
- Rounding: $x_2 = 3$
- Back substitution to first equation: $x_1 = -1$
- Exact solution is $x_1 = -1.002$, $x_2 = 3.001$.
 - Relative error of x_2 is 0.3×10^{-3}
 - Relative error of x_1 becomes 2×10^{-3} .

Gaussian Elimination with Partial Pivoting

- When is pivoting required?
 - In deriving the Gaussian elimination algorithm, we found that a row interchange was needed when one of the pivot elements $a_{kk}^{(k)}$ is 0.
 - When $a_{kk}^{(k)}$ is nonzero but very small in magnitude compared to $a_{jk}^{(k)}$, $k+1 \leq j \leq n$, the magnitude of the multiplier $m_{jk} = \frac{a_{jk}^{(k)}}{a_{kk}^{(k)}}$ will be much larger than 1, which will cause round-off error.
- The partial pivoting strategy
 - The simplest strategy is to select an element in the same column that is below the diagonal and has the largest absolute value;
 - Specifically, we determine p such that $|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|$, and switch the k -th row and p -th row.

3. LU factorization

What is LU factorization?

Definition (LU factorization)

Let $\mathbb{A}$ be a square matrix. An LU factorization of $\mathbb{A}$, into two factors, a lower triangular matrix $\mathbb{L}$ and an upper triangular matrix $\mathbb{U}$, $\mathbb{A} = \mathbb{L}\mathbb{U}$,

Given an $n \times n$ matrix $\mathbb{A}$, we seek matrices

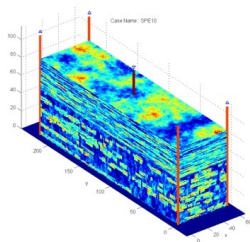
$$\mathbb{L} = \begin{bmatrix} \ell_{1,1} & & & & 0 \\ \ell_{2,1} & \ell_{2,2} & & & \\ \ell_{3,1} & \ell_{3,2} & \ddots & & \\ \vdots & \vdots & \ddots & \ddots & \\ \ell_{n,1} & \ell_{n,2} & \dots & \ell_{n,n-1} & \ell_{n,n} \end{bmatrix}, \mathbb{U} = \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & \dots & u_{1,n} \\ & u_{2,2} & u_{2,3} & \dots & u_{2,n} \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & u_{n-1,n} \\ 0 & & & & u_{n,n} \end{bmatrix}$$

such that $\mathbb{A} = \mathbb{L}\mathbb{U}$.

- In numerical analysis, **LU factorization** (where 'LU' stands for 'lower upper') factors a matrix as the product of a lower triangular matrix and an upper triangular matrix.
- The LU factorization was introduced by the Polish astronomer Tadeusz Banachiewicz in 1938.
- Computers usually solve square systems of linear equations $\mathbf{Ax} = \mathbf{b}$ using the LU factorization, and it is also a key step when inverting a matrix, or computing the determinant of a matrix.

Example: Multiple query problem

- Why do we need the LU decomposition? **Multiple query problem** in reservoir simulation. $-\nabla \cdot (a^\varepsilon(x, \omega) \nabla u^\varepsilon(x, \omega)) = f(x, \theta)$



Does LU factorization exist?

- For example, for a 3-by-3 matrix $\mathbb{A}$, its LU decomposition looks like this:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}.$$

- Without a proper **ordering or permutations** in the matrix, the factorization may fail to materialize. For example, it is easy to verify that $a_{11} = l_{11}u_{11}$. If $a_{11} = 0$, then at least one of l_{11} and u_{11} has to be zero, which implies either $\mathbb{L}$ or $\mathbb{U}$ is singular. This is impossible if $\mathbb{A}$ is nonsingular.
- This is a procedural problem. It can be removed by simply **reordering or pivoting** the rows of $\mathbb{A}$ so that the first element of the permuted matrix is nonzero.

LU decomposition may not exist. We state without proof the following

Existence and Uniqueness of LU Decomposition

不 pivot 的高斯消元不会遇到 0 主元

Let $\mathbb{A}$ be a square matrix of order n . If the first $(n - 1)$ **leading principal minors**^a are non-zero, then $\mathbb{A}$ has a *LU decomposition*. If $\det \mathbb{A} \neq 0$ as well, then the decomposition is unique provided that $\mathbb{L}$ is **unit**, i.e., diagonal elements of $\mathbb{L}$ are 1.

^aThe leading principal submatrix of order k of an $n \times n$ matrix is obtained by deleting the last $n - k$ rows and columns of the matrix. The determinant of a leading principal submatrix is called the leading principal minor.

Doolittle algorithm

We begin with an $n \times n$ matrix $\mathbb{A}$ and search for matrices

$$\mathbb{L} = \begin{bmatrix} \ell_{1,1} & & & & 0 \\ \ell_{2,1} & \ell_{2,2} & & & \\ \ell_{3,1} & \ell_{3,2} & \ddots & & \\ \vdots & \vdots & \ddots & \ddots & \\ \ell_{n,1} & \ell_{n,2} & \dots & \ell_{n,n-1} & \ell_{n,n} \end{bmatrix}, \mathbb{U} = \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & \dots & u_{1,n} \\ & u_{2,2} & u_{2,3} & \dots & u_{2,n} \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & u_{n-1,n} \\ 0 & & & & u_{n,n} \end{bmatrix}$$

such that $\mathbb{A} = \mathbb{L}\mathbb{U}$.

- It turns out that $\mathbb{L}$ and $\mathbb{U}$ are not uniquely determined by assuming $\mathbb{A} = \mathbb{L}\mathbb{U}$. One popular treatment is assuming $\mathbb{L}$ is **unit lower triangular** ($\ell_{i,i} = 1$ for $1 \leq i \leq n$).
- When $\mathbb{U} = \mathbb{L}^T$ so that $\ell_{i,j} = u_{j,i}$, the factorization is called **Cholesky's** factorization. At this time, $\mathbb{A}$ should be real symmetric and positive definite matrix. This factorization has a lot of applications in **probability** and **statistics**.

Doolittle algorithm (cont.)

Input $n, (a_{ij})$

for $k = 1$ to n do

Specify $\ell_{k,k} = 1$

for $j = k$ to n do

$$u_{k,j} = (a_{k,j} - \sum_{s=1}^{k-1} \ell_{k,s} u_{s,j})$$

end do

for $i = k + 1$ to n do

$$\ell_{i,k} = (a_{i,k} - \sum_{s=1}^{k-1} \ell_{i,s} u_{s,k}) / u_{k,k}$$

end do

end do

output $\ell_{i,j}, u_{i,j}$

Question

What is the computational complexity of the Doolittle algorithm?

Question

What is the computational complexity of the Doolittle algorithm?

Proof.

$$\begin{aligned}\sum_{k=1}^n ((n-k+1)2(k-1) + (n-k)(2k-1)) \\ = 1/3n^3 + n^2/2 - 5n/6.\end{aligned}$$



Complexity comparison with Gaussian elimination method

Compared with the Gaussian elimination method, the highest order term in the Doolittle algorithm is smaller. Do little algorithm.

Example

Let $\mathbb{A} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 3 & 1 & 5 \\ -1 & 1 & -5 & 3 \\ 3 & 1 & 7 & -2 \end{pmatrix}$ whose leading principal minors are

All nonzero

1, 1, -2, 2 so that it has LU decomposition. We use the Doolittle algorithm to find its LU decomposition:

$$\mathbb{A} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 3 & 1 & 5 \\ -1 & 1 & -5 & 3 \\ 3 & 1 & 7 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \ell_{21} & 1 & 0 & 0 \\ \ell_{31} & \ell_{32} & 1 & 0 \\ \ell_{41} & \ell_{42} & \ell_{43} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{pmatrix}.$$

Firstly, the first row of U is the same as that of $\mathbb{A}$; while the first column of L is that of $\mathbb{A}$ divided by $u_{11} = a_{11}$.

Example

(Cont'd) We have

$$\mathbb{A} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 3 & 1 & 5 \\ -1 & 1 & -5 & 3 \\ 3 & 1 & 7 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & \ell_{32} & 1 & 0 \\ 3 & \ell_{42} & \ell_{43} & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{pmatrix}.$$

Refer to the second row of U , we have

$$u_{22} = 3 - (2)(1) = 1, \quad u_{23} = 1 - (2)(1) = -1, \quad u_{24} = 5 - (2)(1) = 3.$$

Meanwhile, the second column of L leads to

$$\ell_{32} = \frac{1}{u_{22}}(1 - (-1)(1)) = 2, \quad \ell_{42} = \frac{1}{u_{22}}(1 - (3)(1)) = -2.$$

Example

(Cont'd) We have

$$\mathbb{A} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 3 & 1 & 5 \\ -1 & 1 & -5 & 3 \\ 3 & 1 & 7 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & 2 & 1 & 0 \\ 3 & -2 & \ell_{43} & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 3 \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{pmatrix}.$$

It follows that

$$u_{33} = -5 - (-1)(1) - (2)(-1) = -2, \quad u_{34} = 3 - (-1)(1) - (2)(3) = -2$$

and

$$\ell_{43} = \frac{1}{-2}(7 - (3)(1) - (-2)(-1)) = -1.$$

Example

(Cont'd) Lastly, we arrive at the decomposition

$$\mathbb{A} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 3 & 1 & 5 \\ -1 & 1 & -5 & 3 \\ 3 & 1 & 7 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & 2 & 1 & 0 \\ 3 & -2 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 3 \\ 0 & 0 & -2 & -2 \\ 0 & 0 & 0 & u_{44} \end{pmatrix}$$

whereby

$$u_{44} = -2 - (3)(1) - (-2)(3) - (-1)(-2) = -1.$$

LU decomposition for solving linear system

- We consider to apply the LU factorization to solve linear system $A\mathbf{x} = \mathbf{b}$.
 - 1 Decompose $A = LU$.
 - 2 Solve $L\mathbf{y} = \mathbf{b}$ by forward substitution.
 - 3 Solve $U\mathbf{x} = \mathbf{y}$ by back substitution.
- Actually we have $\mathbf{x} = U^{-1}\mathbf{y} = U^{-1}L^{-1}\mathbf{b} = (LU)^{-1}\mathbf{b} = A^{-1}\mathbf{b}$
- Computational complexity
 - Step 1: $1/3n^3 + n^2/2 - 5n/6$
 - Step 2 and 3: $2n^2$.
- If we need only solve the problem $A\mathbf{x} = \mathbf{b}$ once, the savings are small.
- However, if we need to solve $A\mathbf{x} = \mathbf{b}$ many time with different $\mathbf{b}$, the savings can be huge.

Computing Determinants via LU Decomposition

Given an LU decomposition $A = LU$, then it holds

$$\begin{aligned}\det(A) &= \det(LU) \\ &= \det(L) \det(U) \\ &= \prod_{i=1}^n L_{ii} \prod_{i=1}^n U_{ii}.\end{aligned}$$

Computational complexity

number of flops $\approx 2n$.

浮点计算

4. Stability analysis

Motivation

In the numerical linear equation system,

$$\mathbb{A}\mathbf{x} = \mathbf{b}, \quad (6)$$

we want to study how the error in the input $\mathbf{b}$ propagate to the output $\mathbf{x}$.

- Both the matrix $\mathbb{A}$ and vector $\mathbf{b}$ are input data in (6), and they can suffer from many sources of errors, e.g., rounding errors and measurement error.
- In this section, we assume the matrix $\mathbb{A}$ is exact, and we want to investigate how the error in the input data $\mathbf{b}$ influence the output $\mathbf{x}$.

Problem to solve

Assume that the exact input data is $\mathbf{b}$, and the measured input data is $\mathbf{b} + \delta\mathbf{b}$ due to rounding error or noise, then we are solve

$$\mathbf{A}\tilde{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}. \quad (7)$$

Denote $\tilde{\mathbf{x}} := \mathbf{x} + \delta\mathbf{x}$, then we want to know how the relative error in the output is influence by the relative error in the input data, i.e., can we establish the following inequality,

$$\frac{|\delta\mathbf{x}|}{|\mathbf{x}|} \leq C \frac{|\delta\mathbf{b}|}{|\mathbf{b}|}.$$

To this end, we have to introduce the norm for a vector space.

Let $\mathbb{X}$ be a vector space, for example $\mathbb{X} = \mathbb{R}^n$. $\forall \mathbf{x} \in \mathbb{X}$, i.e., $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$. A map $\|\cdot\| : \mathbb{X} \mapsto \mathbb{R}_0^+$ is a norm on $\mathbb{X}$ if it satisfies

- (Non-negativity) $\|\mathbf{x}\| \geq 0$, $\forall \mathbf{x} \in \mathbb{X}$ and $\|\mathbf{x}\| = 0$ iff $\mathbf{x} = \mathbf{0}$.
- (Linearity) $\|\lambda \mathbf{x}\| = |\lambda| \|\mathbf{x}\|$, $\forall \mathbf{x} \in \mathbb{X}$, $\lambda \in \mathbb{R}$.
- (Triangle inequality.) $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$, $\forall \mathbf{x}, \mathbf{y} \in \mathbb{X}$

Some typical norms are

- Euclidean norm or ℓ_2 -norm: $\|\mathbf{x}\|_2 := \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$.
- ℓ_1 -norm: $\|\mathbf{x}\|_1 := |x_1| + |x_2| + \dots + |x_n|$.
- ℓ_∞ -norm: $\|\mathbf{x}\|_\infty := \max\{|x_1|, |x_2|, \dots, |x_n|\}$.

Matrix norm

- **matrix norm** induced by a given **vector norm**:

$$\|\mathbb{A}\| = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbb{A}\mathbf{x}\|}{\|\mathbf{x}\|} \Rightarrow \|\mathbb{A}\mathbf{x}\| \leq \|\mathbb{A}\| \cdot \|\mathbf{x}\|$$

Maximum possible value

- Special cases of interest are:

- The **1-norm**, $\|\mathbb{A}\|_1 = \max_j \sum_{i=1}^n |a_{ij}|$ Max column sum
- The **∞ -norm**, $\|\mathbb{A}\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$ Max row sum
- The **2-norm** or the **spectral norm**, $\|\mathbb{A}\|_2 = \sigma_1$, the largest singular value.
- The **Frobenius norm**, $\|\mathbb{A}\|_F = \left(\sum_{i=1}^n \sum_{j=1}^n a_{ij}^2 \right)^{\frac{1}{2}}$

Example

If $A = \begin{pmatrix} 2 & -1 \\ 2 & 1 \end{pmatrix}$, evaluate $\|A\|_1$, $\|A\|_2$ and $\|A\|_\infty$.

一个矩阵 A 的奇异值 (singular value),

是 $A^T A$ (或 AA^T) 的非负特征值的算术平方根。

1) $\|A\|_1$ (最大列和)

- 第1列: $|2| + |2| = 4$
- 第2列: $|-1| + |1| = 2$

$$\|A\|_1 = 4$$

2) $\|A\|_\infty$ (最大行和)

- 第1行: $|2| + |-1| = 3$
- 第2行: $|2| + |1| = 3$

$$\|A\|_\infty = 3$$

3) $\|A\|_2$ (谱范数 = 最大奇异值)

$$A^T A = \begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} 2 & -1 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 8 & 0 \\ 0 & 2 \end{pmatrix}$$

最大特征值是 8, 所以

$$\|A\|_2 = \sqrt{8} = 2\sqrt{2}$$

最终答案:

$$\|A\|_1 = 4, \quad \|A\|_2 = 2\sqrt{2}, \quad \|A\|_\infty = 3$$

Quiz

Prove that $\|\mathbb{A}\|_1 = \max_j \sum_{i=1}^n |a_{ij}|$.

1/2.

Let $\eta := \max_j \sum_{i=1}^n |a_{ij}|$. Then by the definition of matrix norm induced by ℓ_1 -norm, we obtain

$$\|\mathbb{A}\|_1 \geq \frac{\|\mathbb{A}\mathbf{e}_j\|_1}{\|\mathbf{e}_j\|_1} = \|\mathbb{A}\mathbf{e}_j\|_1 = \|\mathbf{a}_j\|_1 = \sum_{i=1}^n |a_{ij}|.$$

Here, $\mathbf{e}_j$ is the cononical unit vector in $\mathbb{R}^n$ for $j = 1, \dots, n$, and $\mathbf{a}_j$ denotes the j th column vector of $\mathbb{A}$. Consequently, we derive $\|\mathbb{A}\|_1 \geq \eta$. □

2/2.

To show that $\|\mathbb{A}\|_1 \leq \eta$. Then by the definition of matrix norm induced by ℓ_1 -norm, we obtain

$$\begin{aligned}\|\mathbb{A}\|_1 &= \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbb{A}\mathbf{x}\|_1}{\|\mathbf{x}\|_1} = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\sum_{i=1}^n |\sum_{j=1}^n a_{ij}x_j|}{\|\mathbf{x}\|_1} \\ &\leq \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\sum_{i=1}^n \sum_{j=1}^n |a_{ij}| \cdot |x_j|}{\|\mathbf{x}\|_1} \\ &= \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\sum_{j=1}^n (\sum_{i=1}^n |a_{ij}|) \cdot |x_j|}{\|\mathbf{x}\|_1} \\ &\leq \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\sum_{j=1}^n \eta \cdot |x_j|}{\|\mathbf{x}\|_1} = \eta.\end{aligned}$$

Here, x_j is j th component of the vector $\mathbf{x}$. Consequently, we derive $\|\mathbb{A}\|_1 \leq \eta$. This proves the claim. □

Exercise: self-study

Prove that $\|\mathbb{A}\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$.

1/2.

Let $\eta := \max_i \sum_{j=1}^n |a_{ij}|$. Then by the definition of matrix norm induced by ℓ_∞ -norm, we obtain

$$\|\mathbb{A}\|_\infty \geq \frac{\|\mathbb{A}\mathbf{e}_j\|_\infty}{\|\mathbf{e}_j\|_\infty} = \|\mathbb{A}\mathbf{e}_j\|_\infty \geq \|\mathbf{a}_j\|_1 = \sum_{i=1}^n |a_{ij}|.$$

Here, $\mathbf{e}_j = [\text{sgn}(a_{j1}), \text{sgn}(a_{j2}), \dots, \text{sgn}(a_{jn})]'$ is a column vector generated by the sign function of $\mathbf{a}_j$ for $j = 1, \dots, n$, where $\mathbf{a}_j$ denotes the j th **row** vector of $\mathbb{A}$. Consequently, we derive $\|\mathbb{A}\|_\infty \geq \eta$. □

2/2.

To show that $\|\mathbb{A}\|_\infty \leq \eta$. Then by the definition of matrix norm induced by ℓ_∞ -norm, we obtain

$$\begin{aligned}\|\mathbb{A}\|_\infty &= \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbb{A}\mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\sup_i \left\{ \left| \sum_{j=1}^n a_{ij} x_j \right| \right\}}{\|\mathbf{x}\|_\infty} \\ &\leq \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\sup_i \left\{ \sum_{j=1}^n |a_{ij}| \cdot |x_j| \right\}}{\|\mathbf{x}\|_\infty} \\ &\leq \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\left(\sup_i \sum_{j=1}^n |a_{ij}| \right) \cdot \sup_j \{|x_j|\}}{\|\mathbf{x}\|_\infty} \\ &\leq \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\eta \cdot \sup_j \{|x_j|\}}{\|\mathbf{x}\|_\infty} = \eta.\end{aligned}$$

Here, x_j is j th component of the vector $\mathbf{x}$. Consequently, we derive $\|\mathbb{A}\|_\infty \leq \eta$. This proves the claim. □

Stability/sensitivity analysis and condition number

When we solve $\mathbb{A}\mathbf{x} = \mathbf{b}$, the right hand side may contain error. Thus, our solution may be effected by the error.

$$\blacksquare \mathbb{A}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b} \Rightarrow \mathbb{A}\mathbf{x} + \mathbb{A}\delta\mathbf{x} = \mathbf{b} + \delta\mathbf{b}$$

$$\blacksquare \delta\mathbf{x} = \mathbb{A}^{-1}\delta\mathbf{b} \Rightarrow \|\delta\mathbf{x}\| \leq \|\mathbb{A}^{-1}\| \cdot \|\delta\mathbf{b}\|$$

Using the bounds

$$\blacksquare \|\mathbf{b}\| \leq \|\mathbb{A}\| \cdot \|\mathbf{x}\| \Rightarrow \|\mathbf{x}\| \geq \|\mathbf{b}\|/\|\mathbb{A}\|$$

The relative error in the solution can be bounded by

$$\blacksquare \frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \frac{\|\mathbb{A}^{-1}\| \cdot \|\delta\mathbf{b}\|}{\|\mathbf{x}\|} \leq \frac{\|\mathbb{A}^{-1}\| \cdot \|\delta\mathbf{b}\|}{\|\mathbf{b}\|/\|\mathbb{A}\|} = \|\mathbb{A}\| \cdot \|\mathbb{A}^{-1}\| \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}$$

where the term $\|\mathbb{A}\| \cdot \|\mathbb{A}^{-1}\|$ is called the condition number of the matrix $\mathbb{A}$. Let $\kappa(\mathbb{A}) := \|\mathbb{A}\| \cdot \|\mathbb{A}^{-1}\|$ denote this **condition number**.

Example

- Let us consider a matrix with $\epsilon > 0$ as follows

$$\mathbb{A} = \begin{bmatrix} 1 & 1 + \epsilon \\ 1 - \epsilon & 1 \end{bmatrix}, \quad \mathbb{A}^{-1} = \epsilon^{-2} \begin{bmatrix} 1 & -1 - \epsilon \\ -1 + \epsilon & 1 \end{bmatrix},$$

- If the ∞ -norm is used, we have $\|\mathbb{A}\|_{\infty} = 2 + \epsilon$ and $\|\mathbb{A}^{-1}\|_{\infty} = \epsilon^{-2}(2 + \epsilon)$. By the definition of the condition number, we get $\kappa(\mathbb{A}) = \|\mathbb{A}\| \cdot \|\mathbb{A}^{-1}\| = \frac{(2 + \epsilon)^2}{\epsilon^2} > \frac{4}{\epsilon^2}$.
- If ϵ is very small, say $\epsilon = 0.01$, then $\kappa(\mathbb{A}) \geq 40000$. In such a case, a small relative perturbation in $\mathbf{b}$ may induce a relative perturbation 40000 times greater in the solution of the system $\mathbb{A}\mathbf{x} = \mathbf{b}$.
- We know the determinant of $\mathbb{A}$ is $\det(\mathbb{A}) = \epsilon^2 \approx 0$. So the problem is **ill-conditioned**. Almost singular

2.3 Sparse matrix and iterative solvers

1. sparse matrix

Sparse matrix

- In scientific computing, a **sparse matrix** is a matrix in which most of the elements are zero.
- There is no strict definition regarding the proportion of zero-value elements for a matrix to qualify as sparse but a common criterion is that **the number of non-zero elements is roughly equal to the number of rows or columns**.
- If most of the elements are non-zero, the matrix is considered **dense**.
- The number of zero-valued elements divided by the total number of elements (e.g., $m \times n$ for an $m \times n$ matrix) is sometimes referred to as the **sparsity** of the matrix.
- The concept of sparsity is useful in **combinatorics** and application areas such as **network theory** and **numerical analysis**, which typically have a low density of significant data or connections. **Large sparse matrices** often appear in scientific or engineering applications when **solving partial differential equations (PDEs)**.

Examples: numerical PDE

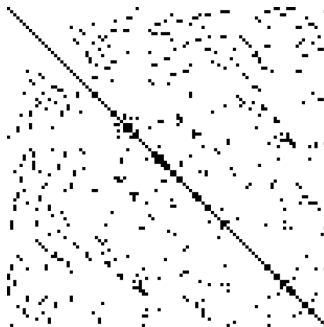


Figure: A sparse matrix obtained when solving a PDE by **finite element method** in 2D domain. The non-zero elements are shown in black.

Example: Graph social network

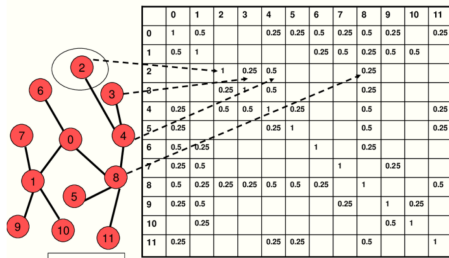


Figure: Illustration of the **graph (social network)** transformation into a sparse matrix where the rows represent instances (vertices) and columns represent neighborhood with weights relative to the distance from the vertex in that row. The diagonal elements have weight 1 (showing that the each vertex is in its own neighborhood). The dashed lines point out neighboring vertices and the corresponding weights for vertex labeled as 2. It has four non-zero elements in its sparse vector representation (1, 0.25, 0.5, 0.25) corresponding to four vertices (labeled in the graph as 2, 3, 4, 8).

Linear equation system with sparse matrix

- In the absence of rounding errors, direct methods would deliver an exact solution. However, **direct methods would be prohibitively expensive** even with the best available computing power.
- Some linear systems are **large** and **sparse**.
 - They can't be stored into an $n \times n$ array, e.g., a 20000×20000 full matrix with double type elements needs $3.2GB$ memory.
 - Number of nonzero elements are $O(n)$.
 - The computational complexity of LU factorization is $O(n^3)$.
 - $\mathbb{A} = \mathbb{L}\mathbb{U}$, where $\mathbb{A}$ is sparse but $\mathbb{L}$ and $\mathbb{U}$ are not, **fill-in** problem.
 - Applications from the **Big Data** Research.
- Iterative methods are often useful for linear problems involving a large number of variables (sometimes of the order of **millions**).
 - **Stationary iterative methods**.
 - **Krylov subspace methods**. Such as the **conjugate gradient (CG)** methods.

Storing a sparse matrix

- When **storing** and **manipulating** sparse matrices on a computer, it is beneficial and often necessary to use specialized algorithms and data structures that take advantage of the **sparse structure of the matrix**.
- Specialized computers have been made for sparse matrices, as they are common in the **machine learning field**.
- Operations using standard **dense-matrix** structures and algorithms are slow and inefficient when applied to large sparse matrices as processing and memory are wasted on the zeros.
- Sparse data is by nature more easily **compressed** and thus requires significantly **less storage**.
- Some very large sparse matrices are **infeasible** to manipulate using standard dense-matrix algorithms.

Storing a sparse matrix (cont.)

- A matrix is typically stored as a **two-dimensional array**. Each entry in the array represents an element a_{ij} of the matrix and is accessed by the two indices i and j . Conventionally, i is the **row index**, numbered from top to bottom, and j is the **column index**, numbered from left to right.
- For an $m \times n$ matrix, the amount of memory required to store the matrix in this format is proportional to $m \times n$ (disregarding the fact that the dimensions of the matrix also need to be stored).
- In the case of a **sparse matrix**, substantial memory requirement reductions can be realized by **storing only the non-zero entries**.
- The **trade-off** is that **accessing the individual elements becomes more complex** and additional structures are needed to be able to recover the original matrix unambiguously.

Storing a sparse matrix (cont.)

- Depending on the **number** and **distribution of the non-zero entries**, different data structures can be used and yield huge savings in memory when compared to the basic approach.
- Formats can be divided into two groups:
 - 1 Those that **support efficient modification**, such as DOK (Dictionary of keys), LIL (List of lists), or COO (Coordinate list). These are typically used to construct the matrices.
 - 2 Those that support **efficient access and matrix operations**, such as CSR (Compressed Sparse Row) or CSC (Compressed Sparse Column).
- The **compressed sparse row (CSR)** or **compressed row storage (CRS)** represents a matrix M by **three (one-dimensional) arrays**, that respectively contain **nonzero values**, the **extents of rows**, and **column indices**. This format allows fast row access and matrix-vector multiplications (Mx).

The CSR format

- The CSR format has been in use since at least the mid-1960s, with the first complete description appearing in 1967.
- The CSR format stores a sparse $m \times n$ matrix M in row form using three (one-dimensional) arrays (V , COL_{INDEX} , ROW_{INDEX}). Let NNZ denote the number of nonzero entries in M .
 - 1 The arrays V and COL_{INDEX} are of length NNZ , and contain the non-zero values and the column indices of those values respectively.
 - 2 The array ROW_{INDEX} is of length $m + 1$ and encodes the index in V and COL_{INDEX} where the given row starts. This is equivalent to $ROW_{INDEX}[j]$ encoding the total number of nonzeros above row j .
 - 3 The last element is NNZ , i.e., the fictitious index in V immediately after the last valid index $NNZ - 1$.

An example

The matrix $\begin{pmatrix} 5 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 6 & 0 & 0 \end{pmatrix}$ is a 4×4 matrix with 4 nonzero elements.

- We have $V = [5, 8, 3, 6]$, $COL_{INDEX} = [0, 1, 2, 1]$, and $ROW_{INDEX} = [0, 1, 2, 3, 4]$, assuming a zero-indexed language.
- To extract a row, we first define: $row_{start} = ROW_{INDEX}[row]$ and $row_{end} = ROW_{INDEX}[row + 1]$. Then, we take slices from V and COL_{INDEX} starting at row_{start} and ending at row_{end} .
- To extract the row 1 (the second row) of this matrix we set $row_{start} = 1$ and $row_{end} = 2$. Then we make the slices $V[1 : 2] = [8]$ and $COL_{INDEX}[1 : 2] = [1]$. We now know that in row 1 we have one element at column 1 with value 8.

The fill-in issue

- Consider a **sparse linear system** $\mathbf{A}\mathbf{x} = \mathbf{b}$. These are often solved by computing the decomposition $\mathbf{A} = \mathbf{L}\mathbf{U}$, with $\mathbf{L}$ lower unittriangular and $\mathbf{U}$ upper triangular. One then solves $\mathbf{L}\mathbf{y} = \mathbf{b}$, $\mathbf{U}\mathbf{x} = \mathbf{y}$, which can be done efficiently because the matrices are triangular.
- For a typical sparse matrix, the LU factors can be **much less sparse** than the original matrix — a phenomenon called **fill-in**. The memory requirements for using a direct solver can then become a **bottleneck** in solving linear systems. One can combat this problem by using fill-reducing reorderings of the matrix's unknowns, such as the **Minimum degree algorithm**.
- An **incomplete factorization** instead seeks triangular matrices $\mathbf{L}$, $\mathbf{U}$ such that $\mathbf{A} \approx \mathbf{L}\mathbf{U}$ rather than $\mathbf{A} = \mathbf{L}\mathbf{U}$. Solving for $\mathbf{L}\mathbf{U}\mathbf{x} = \mathbf{b}$ can be done quickly but does not yield the exact solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$. So, we instead use the matrix $\mathbf{M} = \mathbf{L}\mathbf{U}$ as a **preconditioner** in another iterative solution algorithm such as the **conjugate gradient method** or **GMRES**.

2. Splitting-based Iterative Methods

Definition

Given a matrix equation $\mathbb{A}\mathbf{x} = \mathbf{b}$ where $\mathbb{A}$ is a square matrix. Let $\mathbb{S}$ be an invertible matrix. Then

$$\begin{aligned}\mathbb{S}\mathbf{x} &= (\mathbb{S} - \mathbb{A})\mathbf{x} + \mathbf{b}, \\ \mathbf{x} &= \mathbb{S}^{-1}(\mathbb{S} - \mathbb{A})\mathbf{x} + \mathbb{S}^{-1}\mathbf{b} = (\mathbb{I} - \mathbb{S}^{-1}\mathbb{A})\mathbf{x} + \mathbb{S}^{-1}\mathbf{b}.\end{aligned}$$

Then we have the iterative scheme

$$\mathbf{x}_{i+1} = (\mathbb{I} - \mathbb{S}^{-1}\mathbb{A})\mathbf{x}_i + \mathbb{S}^{-1}\mathbf{b}. \quad (8)$$

The matrix $\mathbb{S}$ is called a *splitting matrix*.

Question

Prove that the sequence $\{\mathbf{x}_n\}$ satisfies

$$\mathbf{x}_k = (\mathbb{I} - \mathbb{S}^{-1}\mathbb{A})^k \mathbf{x}_0 + \sum_{i=0}^{k-1} (\mathbb{I} - \mathbb{S}^{-1}\mathbb{A})^i \mathbb{S}^{-1} \mathbf{b}. \quad (9)$$

In addition, if one subtract the original equation

$$\mathbf{x} = (\mathbb{I} - \mathbb{S}^{-1}\mathbb{A})\mathbf{x} + \mathbb{S}^{-1}\mathbf{b}$$

from (8), we have

$$\mathbf{x}_{i+1} - \mathbf{x} = (\mathbb{I} - \mathbb{S}^{-1}\mathbb{A})(\mathbf{x}_i - \mathbf{x}).$$

Denote $\mathbf{e}_j := \mathbf{x}_j - \mathbf{x}$ as the error at the j th iteration. Then apply this formula repeatedly, we obtain

$$\mathbf{e}_{i+1} = (\mathbb{I} - \mathbb{S}^{-1}\mathbb{A})^{i+1} \mathbf{e}_0. \quad (10)$$

Our objective is to choose $\mathbb{S}$ so that

- (i) $\{\mathbf{x}_i\}$ is easily computed,
- (ii) $\{\mathbf{x}_i\}$ converges rapidly to the solution.

Convergence

Similar to the convergence of geometric series, we have

Theorem

The iterative scheme (8) converges with respect to some vector norm for any $\mathbf{b}$ and initial value $\mathbf{x}_0$ if $\|\mathbf{I} - \mathbf{S}^{-1}\mathbf{A}\| < 1$ where $\|\cdot\|$ is the matrix norm subordinate to that vector norm.

Proof.

Suppose that $\|\mathbf{I} - \mathbf{S}^{-1}\mathbf{A}\| < 1$ for some subordinate matrix norm $\|\cdot\|$. In view of (10),

$$\|\mathbf{x}_{i+1} - \mathbf{x}\| \leq \|\mathbf{I} - \mathbf{S}^{-1}\mathbf{A}\|^{i+1} \|\mathbf{x}_0 - \mathbf{x}\| \rightarrow 0 \text{ as } i \rightarrow \infty.$$



A precise criterion can be given in terms of **spectral radius**. We state without proof the following

Theorem

An iterative scheme

$$\mathbf{x}_{i+1} = \mathbb{G}\mathbf{x}_i + \mathbf{c}$$

converges for any vector $\mathbf{c}$ and any initial value $\mathbf{x}_0$ with respect to some vector norm if and only if $\rho(\mathbb{G}) < 1$, i.e., $\|\mathbb{G}\|_2 < 1$.

Equivalence of norms in finite dimensional spaces

All norms in $\mathbb{R}^n$ are equivalent, i.e., there are positive constants c_1 and c_2 , such that

$$c_1 M(\mathbf{x}) \leq N(\mathbf{x}) \leq c_2 M(\mathbf{x}),$$

where $M(\cdot)$ and $N(\cdot)$ denote any two norms defined over $\mathbb{R}^n$.

Convergence of iterative schemes

Due to the equivalence of norms in finite dimensional spaces, we can study the convergence of the iterative schemes with respect to any norms, which guarantees the convergence of this scheme.

Jacobi iterative method

- In numerical linear algebra, the **Jacobi iterative method** is an algorithm for determining the solutions of a diagonally dominant system of linear equations.
- Given a square system of n linear equations $\mathbb{A}\mathbf{x} = \mathbf{b}$, where:

$$\mathbb{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

- $\mathbb{A}$ is decomposed into a **diagonal component** $\mathbb{D}$, and the remainder $\mathbb{R}$:

$$\mathbb{A} = \mathbb{D} + \mathbb{R}, \mathbb{D} = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{bmatrix}, \mathbb{R} = \begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ a_{21} & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & 0 \end{bmatrix}.$$

Jacobi iterative method (cont.)

- The iterative scheme is $\mathbf{x}^{(k+1)} = \mathbb{D}^{-1}(\mathbf{b} - \mathbb{R}\mathbf{x}^{(k)}) = \mathbb{D}^{-1}\mathbf{b} - \mathbb{D}^{-1}\mathbb{R}\mathbf{x}^{(k)}$, where $\mathbf{x}^{(k)}$ is the k -th approximation or iteration of $\mathbf{x}$ and $\mathbf{x}^{(k+1)}$ is the next or $k+1$ iteration of $\mathbf{x}$.
- The element-based formula is thus:
$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n.$$
 The computation of $x_i^{(k+1)}$ requires each element in $\mathbf{x}^{(k)}$ except itself.
- The iterative scheme is easy to be parallelized.

Jacobi iterative method (cont.)

Theorem (General convergence condition)

The standard convergence condition (for any iterative method) is when the norm of the iteration matrix, i.e., $\|\mathbf{D}^{-1}\mathbf{R}\|$ is less than 1.

Theorem (Strictly diagonally dominant case)

*If the matrix $\mathbf{A}$ is **strictly diagonally dominant**, i.e., $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$, then the Jacobi iterative method is guaranteed to converge in ∞ -norm.*

The Jacobi method sometimes converges even if these conditions are not satisfied.

Example

The matrix

$$\begin{pmatrix} k & 1 & -2 \\ 0 & 5 & -2 \\ 1 & -1 & 3 \end{pmatrix}$$

is strictly diagonally dominant if and only if $|k| > 3$.

Example

Suppose we are given the following linear system:

$$\begin{aligned}10x_1 - x_2 + 2x_3 &= 6, \\ -x_1 + 11x_2 - x_3 + 3x_4 &= 25, \\ 2x_1 - x_2 + 10x_3 - x_4 &= -11, \\ 3x_2 - x_3 + 8x_4 &= 15.\end{aligned}$$

then

$$\mathbb{A} = \begin{bmatrix} 10 & -1 & 2 & 0 \\ -1 & 11 & -1 & 3 \\ 2 & -1 & 10 & -1 \\ 0 & 3 & -1 & 8 \end{bmatrix}, \text{ and } \mathbb{D} = \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 11 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 8 \end{bmatrix}$$

- Suppose we choose $(0, 0, 0, 0)$ as the initial approximation, then the first approximate solution is given by

$$\begin{aligned}x_1 &= 6/10 = 0.6, x_2 = 25/11 = 2.2727, \\ x_3 &= 11/10 = -1.1, x_4 = 15/8 = 1.875.\end{aligned}$$

Example (cont.)

- Using the approximations obtained, the iterative procedure is repeated until the desired accuracy has been reached. The following are the approximated solutions after five iterations.

x_1	x_2	x_3	x_4
0.6	2.27272	-1.1	1.875
1.04727	1.7159	-0.80522	0.88522
0.93263	2.05330	-1.0493	1.13088
1.01519	1.95369	-0.9681	0.97384
0.98899	2.0114	-1.0102	1.02135

- The exact solution of the system is (1, 2, -1, 1).

Gauss-Seidel iterative method

- The Gauss-Seidel method is an iterative technique for solving a square system of n linear equations with unknown $\mathbf{x}$, $\mathbb{A}\mathbf{x} = \mathbf{b}$.
- Then decompose $\mathbb{A}$ into its **lower triangular** component and its **strictly upper triangular** component is given by:

$$\mathbb{A} = \mathbb{L}_* + \mathbb{U}, \mathbb{L}_* = \begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ a_{21} & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \mathbb{U} = \begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}.$$

- The system of linear equations may be rewritten as $\mathbb{L}_*\mathbf{x} = \mathbf{b} - \mathbb{U}\mathbf{x}$

- The Gauss-Seidel method solves the left hand side for $\mathbf{x}$, using previous value for $\mathbf{x}$ on the right hand side, i.e.,
 $\mathbf{x}^{(k+1)} = \mathbb{L}_*^{-1}(\mathbf{b} - \mathbb{U}\mathbf{x}^{(k)})$.
- However, by taking advantage of the triangular form of $\mathbb{L}_*$, the elements of $\mathbf{x}^{(k+1)}$ can be computed sequentially using forward substitution:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n.$$

Some remarks

- The element-wise formula for the Gauss-Seidel method is extremely similar to that of the Jacobi method.
- The computation of $x_i^{(k+1)}$ **uses only the elements of $\mathbf{x}^{(k+1)}$ that have already been computed**, and only the elements of $\mathbf{x}^{(k)}$ that have not yet to be advanced to iteration $k + 1$.
- This means that, like the Jacobi method, **only one storage vector** is required as elements can be overwritten as they are computed, which can be advantageous for very large problems.
- However, unlike the Jacobi method, the computations for each element **cannot be done in parallel**. Furthermore, the values at each iteration are dependent on the order of the original equations.

Theorem (General convergence condition)

*The convergence properties of the Gauss-Seidel method are dependent on the matrix $\mathbb{A}$. The procedure is known to converge if $\mathbb{A}$ is **symmetric positive-definite**, or $\mathbb{A}$ is **strictly diagonally dominant**.*

- The Gauss-Seidel method sometimes converges even if these conditions are not satisfied.
- The iterative scheme is hard to parallelize.

Example

Let's consider the same example.

$$\begin{aligned}10x_1 - x_2 + 2x_3 &= 6, \\ -x_1 + 11x_2 - x_3 + 3x_4 &= 25, \\ 2x_1 - x_2 + 10x_3 - x_4 &= -11, \\ 3x_2 - x_3 + 8x_4 &= 15.\end{aligned}$$

Solving for x_1 , x_2 , x_3 and x_4 gives:

$$\begin{aligned}x_1 &= x_2/10 - x_3/5 + 3/5, \\ x_2 &= x_1/11 + x_3/11 - 3x_4/11 + 25/11, \\ x_3 &= -x_1/5 + x_2/10 + x_4/10 - 11/10, \\ x_4 &= -3x_2/8 + x_3/8 + 15/8.\end{aligned}$$

Again, we choose $(0, 0, 0, 0)$ as the initial guess, then the first update is

$$\begin{aligned}x_1 &= 3/5 = 0.6, \\ x_2 &= (3/5)/11 + 25/11 = 3/55 + 25/11 = 2.3272, \\ x_3 &= -(3/5)/5 + (2.3272)/10 - 11/10 = -0.9873, \\ x_4 &= -3(2.3272)/8 + (-0.9873)/8 + 15/8 = 0.8789.\end{aligned}$$

Example (cont.)

- Using the approximations obtained, the iterative procedure is repeated until the desired accuracy has been reached. The following are the approximated solutions after four iterations.

x_1	x_2	x_3	x_4
0.6	2.32727	-0.987273	0.878864
1.03018	2.03694	-1.01446	0.984341
1.00659	2.00356	-1.00253	0.998351
1.00086	2.0003	-1.00031	0.99985

- The exact solution of the system is $(1, 2, -1, 1)$.
- **If converge**, the convergence speed of Gauss-Seidel method is faster than the Jacobi method.

2.4 Singular Value Decomposition (SVD) and Low-rank approximation

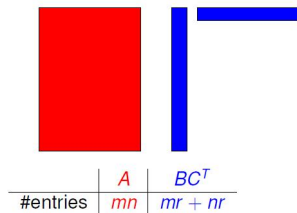
1. Definition and Computation

Rank and matrix factorizations

Lemma

A matrix $\mathbb{A} \in \mathbb{R}^{m \times n}$ of rank r admits a factorization of the form $\mathbb{A} = \mathbb{B}\mathbb{C}^T$, where $\mathbb{B} \in \mathbb{R}^{m \times r}$ and $\mathbb{C} \in \mathbb{R}^{n \times r}$.

- We say that $\mathbb{A}$ has **low rank** if $\text{rank}(\mathbb{A}) \ll m, n$.
- Illustration of low-rank factorization:



- Generically (and in most applications), $\mathbb{A}$ has full rank, that is, $\text{rank}(\mathbb{A}) = \min\{m, n\}$.
- We aim to approximate $\mathbb{A}$ by a **low-rank matrix**.

Why low-rank representation matters?

Example (Matrix-Vector multiplication)

Let $\mathbb{A} \in \mathbb{R}^{n \times n}$ be a matrix of rank k with $n = 100000$ and $k = 5$. Let $\mathbf{x} \in \mathbb{R}^n$ be a vector. We want to compute

$$\mathbb{A}\mathbf{x}.$$

If computing directly, then the computational complexity is n^2 , which is undoable.

Assume that $\mathbb{A}$ admits a factorization of the form $\mathbb{A} = \mathbb{B}\mathbb{C}^T$, where $\mathbb{B} \in \mathbb{R}^{n \times r}$ and $\mathbb{C} \in \mathbb{R}^{n \times r}$. Here, $n = 100000$ and $k = 5$. Then we can compute

$$\mathbb{A}\mathbf{x} = \mathbb{B}\mathbb{C}^T\mathbf{x} = \mathbb{B}\left(\mathbb{C}^T\mathbf{x}\right).$$

In this manner, the computational complexity is $2nr \ll n^2$.

Rank and matrix factorizations

Theorem (Singular Value Decomposition (SVD))

Let $\mathbb{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$. Then there are orthonormal matrices $\mathbb{U} \in \mathbb{R}^{m \times m}$ and $\mathbb{V} \in \mathbb{R}^{n \times n}$ such that $\mathbb{A} = \mathbb{U}\Sigma\mathbb{V}^T$, where

$$\Sigma = \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \\ 0 & \dots & 0 \end{bmatrix} \in \mathbb{R}^{m \times n}$$

and $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$.

- $\sigma_1, \dots, \sigma_n$ are called **singular values**.
- $\mathbb{U} = [\mathbf{u}_1, \dots, \mathbf{u}_m]$. $\mathbf{u}_1, \dots, \mathbf{u}_m$ are called **left singular vectors**.
- $\mathbb{V} = [\mathbf{v}_1, \dots, \mathbf{v}_n]$. $\mathbf{v}_1, \dots, \mathbf{v}_n$ are called **right singular vectors**.
- $\mathbb{A}\mathbf{v}_i = \sigma_i\mathbf{u}_i$, $\mathbb{A}^T\mathbf{u}_i = \sigma_i\mathbf{v}_i$ for $i = 1, \dots, n$.
- Singular values are always uniquely defined by $\mathbb{A}$.

SVD: Sketch of proof (Induction over n)

- $n = 1$ trivial.
- For $n > 1$, let

$$\mathbf{v}_1 = \arg \max_{\mathbf{v} \in \mathbb{R}^n} \{\|\mathbb{A}\mathbf{v}\|_2 : \|\mathbf{v}\|_2 = 1\}.$$

Set $\sigma_1 := \|\mathbb{A}\mathbf{v}_1\|_2$ and $\mathbf{u}_1 := \mathbb{A}\mathbf{v}_1/\sigma_1$. By definition, $\mathbb{A}\mathbf{v}_1 = \sigma_1\mathbf{u}_1$.

- After completion to orthonormal matrices $\mathbb{U}_1 = [\mathbf{u}_1, \mathbb{U}_\perp] \in \mathbb{R}^{m \times m}$ and $\mathbb{V}_1 = [\mathbf{v}_1, \mathbb{V}_\perp] \in \mathbb{R}^{n \times n}$:

$$\mathbb{U}_1^T \mathbb{A} \mathbb{V}_1 = \begin{bmatrix} \mathbf{u}_1^T \mathbb{A} \mathbf{v}_1 & \mathbf{u}_1^T \mathbb{A} \mathbb{V}_\perp \\ \mathbb{U}_\perp^T \mathbb{A} \mathbf{v}_1 & \mathbb{U}_\perp^T \mathbb{A} \mathbb{V}_\perp \end{bmatrix} = \begin{bmatrix} \sigma_1 & \mathbf{w}^T \\ 0 & \mathbb{A}_1 \end{bmatrix}$$

with $\mathbf{w} = \mathbb{V}_\perp^T \mathbb{A}^T \mathbf{u}_1$ and $\mathbb{A}_1 = \mathbb{U}_\perp^T \mathbb{A} \mathbb{V}_\perp$. $\|\cdot\|_2$ is invariant under orthonormal transformations.

- $\sigma_1 = \|\mathbb{A}\|_2 = \|\mathbb{U}_1^T \mathbb{A} \mathbb{V}_1\|_2 = \left\| \begin{bmatrix} \sigma_1 & \mathbf{w}^T \\ 0 & \mathbb{U}_\perp^T \mathbb{A} \mathbb{V}_\perp \end{bmatrix} \right\|_2 \geq \sqrt{\sigma_1^2 + \|\mathbf{w}\|_2^2}.$

Hence $\mathbf{w} = \mathbf{0}$.

- Proof completed by applying induction to $\mathbb{A}_1$.

Exercise

Can you prove that

$$\left\| \begin{bmatrix} \sigma_1 & \mathbf{w}^T \\ \mathbf{0} & \mathbf{U}_\perp^T \mathbf{A} \mathbf{V}_\perp \end{bmatrix} \right\|_2 \geq \sqrt{\sigma_1^2 + \|\mathbf{w}\|_2^2}.$$

Computation of the SVD

- $\mathbb{A}^T \mathbb{A}$ is an $n \times n$ **symmetric** matrix and **positive semidefinite** since

$$\mathbf{x}^T (\mathbb{A}^T \mathbb{A}) \mathbf{x} = (\mathbb{A} \mathbf{x})^T (\mathbb{A} \mathbf{x}) \geq 0.$$

- Denote the eigenvalues of $\mathbb{A}^T \mathbb{A}$ in a **non-increasing** order by $\sigma_1^2 \geq \sigma_2^2 \geq \dots \geq \sigma_n^2$ such that $\sigma_1^2, \sigma_2^2, \dots, \sigma_r^2$ are positive and $\sigma_{r+1}^2, \sigma_{r+2}^2, \dots, \sigma_n^2$ are 0.
For symmetric or PSD matrix: Rank = # of nonzero eigenvalues
- Let $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\}$ be an **orthonormal** set of eigenvectors for $\mathbb{A}^T \mathbb{A}$ so that

$$\mathbb{A}^T \mathbb{A} \mathbf{v}_i = \sigma_i^2 \mathbf{v}_i$$

- Then

$$\|\mathbb{A} \mathbf{v}_i\|_2^2 = \mathbf{v}_i^T \mathbb{A}^T \mathbb{A} \mathbf{v}_i = \mathbf{v}_i^T \sigma_i^2 \mathbf{v}_i = \sigma_i^2.$$

$\mathbf{v}_i$ is unit vector

Computation of the SVD (cont.)

- Thus

$$\mathbb{A}\mathbf{v}_i = \mathbf{0} \quad \text{when} \quad i \geq r + 1$$

- Observe that

$$r = \text{rank}(\mathbb{A}^T \mathbb{A}) \leq \min\{\text{rank}(\mathbb{A}^T), \text{rank}(\mathbb{A})\} \leq \min\{m, n\}$$

- We form an $n \times n$ matrix $\mathbb{V}^T$ whose rows are $\{\mathbf{v}_1^T; \mathbf{v}_2^T; \dots; \mathbf{v}_n^T\}$ and define

$$\mathbf{u}_i = \sigma_i^{-1} \mathbb{A} \mathbf{v}_i \quad (1 \leq i \leq r).$$

- Note that

$$\mathbb{A} \mathbf{v}_i = \sigma_i \mathbf{u}_i, \quad \mathbb{A}^T \mathbf{u}_i = \sigma_i \mathbf{v}_i \quad \text{for } i = 1, \dots, n.$$

$$\mathbf{u}_i^T \mathbf{u}_j = \sigma_i^{-1} (\mathbb{A} \mathbf{v}_i)^T \sigma_j^{-1} (\mathbb{A} \mathbf{v}_j) = (\sigma_i \sigma_j)^{-1} (\mathbf{v}_i^T \mathbb{A}^T \mathbb{A} \mathbf{v}_j) = \delta_{ij}$$

Thus, the $\mathbf{u}_i$'s form an **orthonormal** system for $1 \leq i, j \leq r$.

Computation of the SVD (cont.)

- Select additional vectors $\mathbf{u}_i$, $r + 1 \leq i \leq m$ so that $\{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_m\}$ is an orthonormal base for $\mathbb{R}^m$.
- Let $\mathbb{U}$ be the $m \times m$ matrix, whose columns are $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_m$.
- Let Σ be the $m \times n$ matrix, having $\sigma_1, \sigma_2, \dots, \sigma_r$ on its diagonal and 0's elsewhere.
- Then

$$(\mathbb{U}^T \mathbb{A} \mathbb{V})_{ij} = \mathbf{u}_i^T \mathbb{A} \mathbf{v}_j = \sigma_{ij}$$

Hence $\mathbb{A} = \mathbb{U} \Sigma \mathbb{V}^T$.

- The numbers $\sigma_1, \sigma_2, \dots, \sigma_n$ are called the **singular values** of $\mathbb{A}$, also the nonnegative square roots of the eigenvalues of $\mathbb{A}^T \mathbb{A}$.

Basic properties of the SVD

- $r = \text{rank}(\mathbb{A})$ is number of nonzero singular values of $\mathbb{A} \in \mathbb{R}^{m \times n}$.
- Then, $\mathbb{A} = \mathbf{U}\Sigma\mathbf{V}^T$, with

$$\Sigma = \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \\ 0 & \dots & 0 \end{bmatrix} \in \mathbb{R}^{m \times n}$$

and $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 = \sigma_{r+1} = \dots = \sigma_n$.

- The singular value decomposition can be written as $\mathbb{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$,
where $r \leq \{m, n\}$.

Kernel and range

Definition

Let $\mathbb{A} \in \mathbb{R}^{m \times n}$ be a matrix, then the kernel of $\mathbb{A}$ is

$$\ker(\mathbb{A}) := \{\mathbf{v} \in \mathbb{R}^n : \mathbb{A}\mathbf{v} = \mathbf{0}\}.$$

The range of $\mathbb{A}$ is

$$\text{Range}(\mathbb{A}) := \{\mathbb{A}\mathbf{v} : \mathbf{v} \in \mathbb{R}^n\}.$$

It is well known that $\ker(\mathbb{A})$ is a subspace of $\mathbb{R}^n$ and $\text{Range}(\mathbb{A})$ is a subspace of $\mathbb{R}^m$.

Question

Let the singular value decomposition can be written as $\mathbb{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$,

where $r \leq \{m, n\}$. What is $\ker(\mathbb{A})$ and $\text{range}(\mathbb{A})$?

SVD: Computation (for small or moderate dense matrices)

- In most applications, vectors $\mathbf{u}_{n+1}, \dots, \mathbf{u}_m$ are not of interest. By omitting these vectors one obtains the following variant of the SVD

Theorem (Economy size SVD)

Let matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$. Then, there is a matrix $\mathbf{U} \in \mathbb{R}^{m \times n}$ with orthonormal columns and a orthonormal matrix $\mathbf{V} \in \mathbb{R}^{n \times n}$ such that $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \end{bmatrix} \in \mathbb{R}^{n \times n}$$

and $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$.

SVD: Computation (cont.)

- Computed by MATLAB's $[\mathbf{U}, \mathbf{S}, \mathbf{V}] = \text{svd}(\mathbf{A}, 'econ')$.
- Complexity: memory $O(mn)$.
- Computing the SVD of an $m \times n$ matrix has complexity $O(mn \min(n, m))$. Since this is super-linear in the size of the data, it becomes computationally expensive for large data sets.

Norms: Spectral and Frobenius norm

Given SVD $\mathbb{A} = \mathbb{U}\Sigma\mathbb{V}^T$, one defines:

- **Spectral norm:** $\|\mathbb{A}\|_2 = \sigma_1$ and
- **Frobenius norm:** $\|\mathbb{A}\|_F = \sqrt{\sigma_1^2 + \cdots + \sigma_n^2}$.

Basic properties:

- $\|\mathbb{A}\|_2 = \max\{\|\mathbb{A}\mathbf{v}\|_2 : \|\mathbf{v}\|_2 = 1\}$
- $\|\cdot\|_2$ and $\|\cdot\|_F$ are both matrix norms.
- $\|\cdot\|_2$ and $\|\cdot\|_F$ are both unitarily invariant, that is $\|\mathbb{Q}\mathbb{A}\mathbb{Z}\|_2 = \|\mathbb{A}\|_2$ and $\|\mathbb{Q}\mathbb{A}\mathbb{Z}\|_F = \|\mathbb{A}\|_F$ for any orthogonal matrices $\mathbb{Q}$ and $\mathbb{Z}$.
- $\|\mathbb{A}\|_2 \leq \|\mathbb{A}\|_F \leq \|\mathbb{A}\|_2 \sqrt{r}$.

Exercise

Let $Q \in \mathbb{R}^{m \times m}$ and $Z \in \mathbb{R}^{n \times n}$ be orthogonal matrices and let $A \in \mathbb{R}^{m \times n}$.
Can you prove that

$$\|QAZ\|_2 = \|A\|_2.$$

Example

Find a singular-value decomposition for the matrix

$$\begin{bmatrix} 0 & -1.6 & 0.6 \\ 0 & 1.2 & 0.8 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- Following the proof of the theorem, we have

$$\mathbb{A}^T \mathbb{A} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Eigenvalues: 0 4 1
Rank = 2

Put $\sigma_1 = 1$, $\sigma_2 = 2$ and $\sigma_3 = 0$, and form the matrix $\mathbb{V}^T$

$$\mathbb{V}^T = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

A simple example (cont.)

- Then

$$\mathbf{u}_1 = \mathbb{A} \mathbf{v}_1 = (0.6, 0.8, 0, 0)^T$$

$$\mathbf{u}_2 = \frac{1}{2} \mathbb{A} \mathbf{v}_2 = (-0.8, 0.6, 0, 0)^T$$

- The simplest choice for $\mathbf{u}_3$ and $\mathbf{u}_4$ is

$$\mathbf{u}_3 = (0, 0, 1, 0)^T$$

$$\mathbf{u}_4 = (0, 0, 0, 1)^T$$

The SVD of the given matrix is

$$\begin{bmatrix} 0 & -1.6 & 0.6 \\ 0 & 1.2 & 0.8 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.8 & 0 & 0 \\ 0.8 & 0.6 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ \hline 1 & 0 & 0 \end{bmatrix}$$

SVD for Complex Matrix

Theorem (Singular Value Decomposition (SVD))

A **complex matrix** $A \in \mathbb{C}^{m \times n}$ with $m \geq n$. Then, there are complex **unitary** matrices $U \in \mathbb{C}^{m \times m}$ and $V \in \mathbb{C}^{n \times n}$ such that $A = U\Sigma V^*$, where

$$\Sigma = \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \\ 0 & \dots & 0 \end{bmatrix} \in \mathbb{R}^{m \times n}$$

and $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$. Here V^* is the **conjugate transpose** of V .

The proof of this theorem is the same as the real matrix case.

Best low-rank approximation

low-rank approximation

low-rank approximation is a minimization problem, in which the cost function measures the fit between a given matrix (the data) and an approximating matrix (the optimization variable), subject to a constraint that the approximating matrix has a reduced rank.

Given

- $A \in \mathbb{R}^{m \times n}$
- desired rank $k \in \mathbb{N}$, and $k \ll m$, $k \ll n$

We want to find a best approximating matrix, s.t.,

$$\min_{\substack{B \in \mathbb{R}^{m \times n} \\ \text{rank}(B) \leq k}} \|A - B\| \quad (11)$$

Best low-rank approximation

- Problem (11) has analytical solution in terms of SVD of the matrix A .

Best low-rank approximation

- Consider $k < n$ and let

Truncated SVD

$$\mathbb{U}_k := [\mathbf{u}_1, \dots, \mathbf{u}_k], \quad \Sigma_k := \text{diag}(\sigma_1, \dots, \sigma_k), \quad \mathbb{V}_k := [\mathbf{v}_1, \dots, \mathbf{v}_k],$$

- Then, we define $\mathbb{A}_k := \mathbb{U}_k \Sigma_k \mathbb{V}_k^T$, which has rank at most k . For any unitarily invariant norm $\|\cdot\|$:

这里的 $\|\cdot\|$ 就是你用来衡量“逼近得好不好”的指标。

不同 norm = 不同“好”的定义。

$$\|\mathbb{A} - \mathbb{A}_k\| = \|\text{diag}(0, \dots, 0, \sigma_{k+1}, \dots, \sigma_n)\|$$

- In particular, for spectral norm and the Frobenius norm

$$\|\mathbb{A} - \mathbb{A}_k\|_2 = \sigma_{k+1}, \quad \|\mathbb{A} - \mathbb{A}_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_n^2}.$$

Nearly equal if and only if singular values decay sufficiently quickly.

Best low-rank approximation

Theorem (Matrix approximation lemma)

在所有 $\text{rank} \leq k$ 的矩阵里，截断 SVD 得到的 A_k 是高 A 最近的 (用任何酉不变范数来量距离)。

Let matrix $A \in \mathbb{R}^{m \times n}$ with $m \geq n$. Then,

$$\|A - A_k\| = \min\{\|A - B\| : B \in \mathbb{R}^{m \times n} \text{ has rank at most } k\}$$

holds for any unitarily invariant norm $\|\cdot\|$.

- Proof for $\|\cdot\|_2$: For any $B \in \mathbb{R}^{m \times n}$ of rank $\leq k$, $\text{kernel}(B)$ has dimension $\geq n - k$. Hence, there exists $w \in \text{kernel}(B) \cap \text{range}(V_{k+1})$ with $\|w\|_2 = 1$. Then,

$$\begin{aligned}\|A - B\|_2^2 &\geq \|(A - B)w\|_2^2 = \|U_{k+1}\Sigma_{k+1}V_{k+1}^T w\|_2^2 \\ &= \sum_{j=1}^{k+1} \sigma_j |v_j^T w|^2 \geq \sigma_{k+1} \sum_{j=1}^{k+1} |v_j^T w|^2 = \sigma_{k+1}.\end{aligned}$$

Theorem 23 is also referred to as [Eckart-Young-Mirsky theorem](#). This problem was originally solved by Erhard Schmidt in the finite dimensional context or integral operators (although his methods easily generalize to arbitrary compact operator on Hilbert spaces) and later rediscovered by C. Eckart and G. Young. L. Mirsky generalized the result to arbitrary unitarily invariant norms.

2.Application: Image Compression with SVD

Problem setting

- An image is an array, or a matrix, of square pixels (picture elements) arranged in columns and rows.
 - 1 In a (8-bit) **greyscale image** is represented with an M-by-N array, where each array (each picture element) has an assigned intensity that ranges from 0 to 255.
 - 2 An **RGB image** is represented with an M-by-N-by-3 array where each 3-vector corresponds to the red, green, and blue intensities of each pixel.
- What is the purpose of transforming the matrix $\mathbb{A}$ into $\mathbf{U}\Sigma\mathbf{V}^T$?
- **We want to approximate the $m \times n$ matrix $\mathbb{A}$ by using far fewer entries than in the original matrix.** By using the rank of a matrix we remove the redundant information (the dependant entries) when the matrix rank $r \ll \min\{m, n\}$.
- Recall that

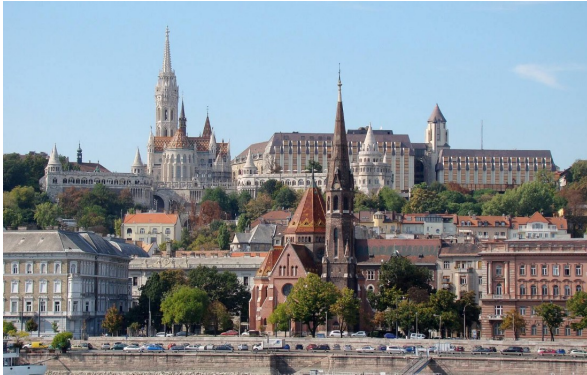
$$\mathbb{A} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^T + 0 \mathbf{u}_{r+1} \mathbf{v}_{r+1}^T + \cdots$$

- Adding on the dependant terms where the singular values are equal to zero does not effect the image. The terms at the end of the equation zero out leaving us with

$$\mathbb{A} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^T.$$

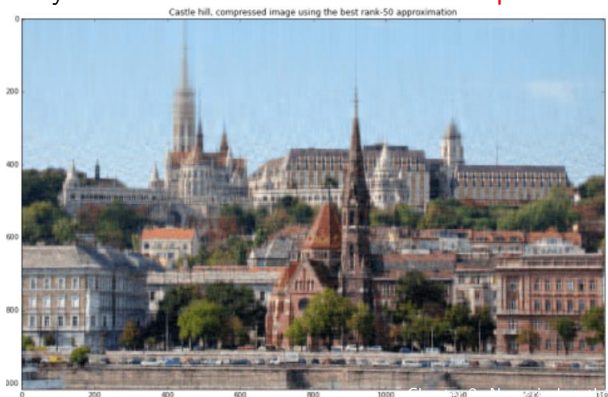
- We can further approximate the matrix by leaving off more singular terms of the matrix $\mathbb{A}$.
- The singular values are arranged in decreasing order. **If the singular values decay very fast**, the last terms will have the least affect on the overall image. Thus, we can only keep the first few singular values and approximate the matrix $\mathbb{A}$ by the corresponding low-rank approximations.

- For demonstration purposes we have chosen an image of the Castle hill in Budapest



Performance 1.

- We **normalize** the intensity values in each pixel (divided the values by 255). Then, separate the grayscale values for each color, i.e. we **work with 3 independent** 2-dimensional matrices, and **perform an SVD-decomposition** for each colour separately.
- We pick $r = 50$, i.e. we construct the best rank-50 approximations of the intensity matrices for each colour. **The compression rate is 8%.**



Performance 2.

- We repeat the experiments using $r = 200$. We construct the best rank-200 approximations of the intensity matrices for each colour. The compression rate is 32.2%.



- Using SVD for image compression can be a very useful tool to save storage space. We were able to get an image that is indistinguishable from the original image, but only using 32% of the original storage space.
- The compression rate depends on the features of the image, such as smoothness, having many edges, etc.
- The SVD is a very powerful mathematical tool with applications in many fields, like image compression that we have just covered briefly, but there are many others.
- SVD decomposition is generally a good choice when one has to compress large dataset (that is, reduce their dimensions) in such a way that the inner structure and correspondence relations between the data points are preserved in some way.

3. Application: Matrix Completion with Low-rank approximation

Problem setting

- **Matrix completion** is the task of filling in the missing entries of a partially observed matrix, which is equivalent to performing data imputation in statistics.
- A wide range of datasets are naturally organized in matrix form. One example is the movie-ratings matrix in **Netflix problem**. Given a ratings matrix in which each entry (i, j) represents the rating of movie j by customer i , if customer i has watched movie j and is otherwise missing, we would like to predict the remaining entries in order to make good recommendations to customers on what to watch next.

				
Alice	4			4
Bob		5	4	
Joe		5		
Sam	5			

- Without any restrictions on the number of degrees of freedom in the completed matrix this problem is **underdetermined** since the hidden entries could be assigned arbitrary values.
- Thus we require some assumption on the matrix to create a well-posed problem, such as **assuming it has maximal determinant, is positive definite**, or is **low-rank**.
- For example, one may assume the matrix has **low-rank structure**, and then seek to find the lowest rank matrix or, if the rank of the completed matrix is known, a matrix of rank k that matches the known entries.
- The matrix completion problem is in general *NP*-hard, but under additional assumptions there are efficient algorithms that achieve exact reconstruction with high probability.

Assume that data matrix is modeled by **low rank k** . Two popular approaches to deal with missing entries:

- 1 Impute data (insert 0 or row/column means in missing entries). Apply SVD to get **best low-rank approximation** $\mathbb{B}\mathbb{C}^T$ of imputed data matrix.
- 2 Find **rank- k matrix** $\mathbb{X}$ that fits known entries best; measured in (weighted) Euclidean norm.
 - The mathematical formulation of this problem is as follows:

$$\begin{aligned} \min_{\mathbb{X}} \quad & \text{rank}(\mathbb{X}) \\ \text{s. t.} \quad & \mathbb{X}_{ij} = \mathbb{M}_{ij} \quad \forall i, j \in E \end{aligned}$$

- Predict unknown entries from $\mathbb{X}$.

Candès proved that with assumptions on the sampling of the observed entries and sufficiently many sampled entries this problem has a unique solution with high probability.

Most of the real-world social networks have low-rank distance matrices.

Criminal networks are a good example of such networks. Low-rank Matrix Completion can be used to recover these unobserved distances.

2.5 Conclusion

Summary

- Using preexisting linear algebra subroutines to perform basic operations such as **vector-vector**, **matrix-vector**, **matrix-matrix** multiplication can save time and yield good performance.
- For small sized ($\mathcal{O}(100)$) or moderate sized ($\mathcal{O}(10000)$) problems, in the absence of roundoff error **direct methods** (e.g., Gaussian elimination and LU Decomposition) would yield the exact solution within a finite number of steps.
- Applications of the **SVD** include computing the **pseudoinverse**, **matrix approximation**, and **determining the rank, range, and null space of a matrix**. It is also extremely useful in all areas of science, engineering, and statistics, such as **signal processing**, **least squares fitting of data**, and **image compression**.